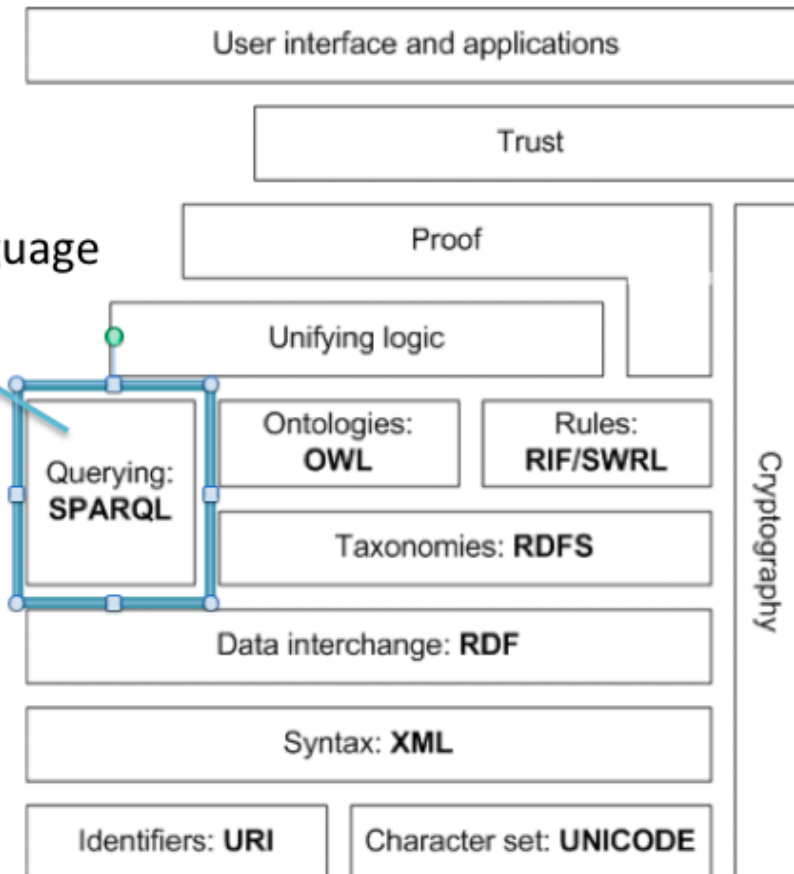


SPARQL_1

Declarative query language



Semantic Web Stack
Berners-Lee (2006)

SPARQL это язык запросов Semantic Web.

Он позволяет:

- Получать значения *из структурированных данных*;
- Исследовать данные с помощью запросов к *неизвестным* отношениям;
- Выполнять *сложные джойны* на разрозненных базах данных в едином простом запросе;
- *Преобразовывать данные RDF* из одного словаря в другой.

Структура запроса SPARQL

Запрос SPARQL содержит (в порядке следования):

Декларации Префиксов (для сокращения URI),

Определение набора данных (указывает граф(ы) RDF, к которым строится запрос)

Тип результата, (определяет, какая информация будет возвращаться из запроса)

Шаблон запроса, (указывает, что искать в основном наборе данных)

Модификаторы запроса (нарезки, упорядочения, перестановки результатов запроса)

Декларации Префиксов

PREFIX foo: <<http://example.com/resources/>>

...

Определение набора данных

FROM ...

Тип результата

SELECT ...

Шаблон запроса,

WHERE {

...

}

Модификаторы запроса

ORDER BY ...

SPARQL & Точки доступа

Запросы SPARQL выполняются на наборах RDF данных, состоящих из RDF графов.

- *SPARQL endpoint* принимает запросы и возвращает результаты через HTTP.
 - *Generic endpoints* запрашивают любой веб-доступный набор данных RDF
 - *Specific endpoints* жестко связаны с определенными наборами данных
- Результаты запросов SPARQL могут быть возвращены в различных форматах:
 - *XML*.
 - *JSON*.
 - *CSV/TSV*.
 - *RDF*.
 - *HTML*.

SPARQL 1.0

- SPARQL 1.0 стал стандартом в январе 2008 года, и включал:
- **SPARQL 1.0 Query Language** -для сопоставления шаблонов с данными RDF
- **SPARQL 1.0 Protocol** -для отправки запросов по HTTP
- **SPARQL Results XML Format**
- **SPARQL Results JSON Format**

SPARQL 1.1

SPARQL 1.1 стал стандартом в марте 2013 года, и включает:

Обновленные версии 1.1 **SPARQL Query** и **SPARQL Protocol**

SPARQL 1.1 Update - для вставки, удаления, модификации данных RDF

SPARQL 1.1 Graph Store HTTP Protocol - использование операций HTTP для управления множеством графов RDF.

SPARQL 1.1 Service Descriptions - описывает возможности SPARQL endpoints

SPARQL 1.1 Entailments - как совместить логический вывод со SPARQL

SPARQL 1.1 Basic Federated Query - запросы к нескольким endpoints одновременно

Результаты запроса SPARQL в форматах CSV/TSV

RDF Term

Определение: RDF Term

Пусть

I - множество всех IRIs.

RDF-L - множество всех RDF литералов

RDF-B - множество всех пустых узлов (blank nodes) в графах RDF

Множество RDF Terms, RDF-T =

$I \cup \text{RDF-L} \cup \text{RDF-B}.$

Терминология Sparql

- 1) **Triple patterns (шаблоны триплетов)** похожи на триплеты RDF, но subject, predicate и object могут быть переменными (?variable).
- 2) Большинство форм SPARQL запроса содержат множество шаблонов триплетов под названием *basic graph pattern (BGP)* – базовый графовый шаблон.
- 3) Базовый графовый шаблон (**то, что стоит в скобке WHERE**) соответствует подграфу данных RDF, когда RDF terms (RDFтермы) из этого подграфа могут быть подставлены на место переменных и результирующий RDF граф будет эквивалентен подграфу данных.

Очень простой пример запроса

Набор данных:

```
<http://example.org/book/book1>  
  <http://purl.org/dc/elements/1.1/title> "SPARQL  
  Tutorial" .
```

Запрос:

```
SELECT ?title
```

```
WHERE
```

```
{  
  <http://example.org/book/book1>  
  <http://purl.org/dc/elements/1.1/title> ?title .  
}
```

Очень простой пример запроса

Запрос состоит из 2 частей:

- 1) Пункт **SELECT** определяет переменные, которые появляются в результатах запроса (**?title**)
- 2) Пункт **WHERE** предоставляет **basic graph pattern (BGP)** базовый графовый шаблон, который надо сравнивать с графом данных .

Базовый графовый шаблон :

<http://example.org/book/book1>

<http://purl.org/dc/elements/1.1/title> ?title .

Result sets (множества результатов)

title
"SPARQL Tutorial"

- 1) Результирующие наборы показаны в табличной форме.
- 2) «Связыванием» **'binding'** является пара (переменная, терм RDF).
- 3) В этом результирующем наборе имеется одна переменная: **title** (показано, как заголовок столбца).
 - 3) Каждое решение показано как одна **строка** в теле таблицы.
 - 4) Здесь есть одно решение, в котором переменная **title** связана с литералом "SPARQL Tutorial".

Множественные соответствия

- Результатом запроса является **последовательность решений**, соответствующая тому, каким образом графовый шаблон запроса соответствует набору данных.
Может быть ноль, один или несколько решений запроса.

Множественные соответствия

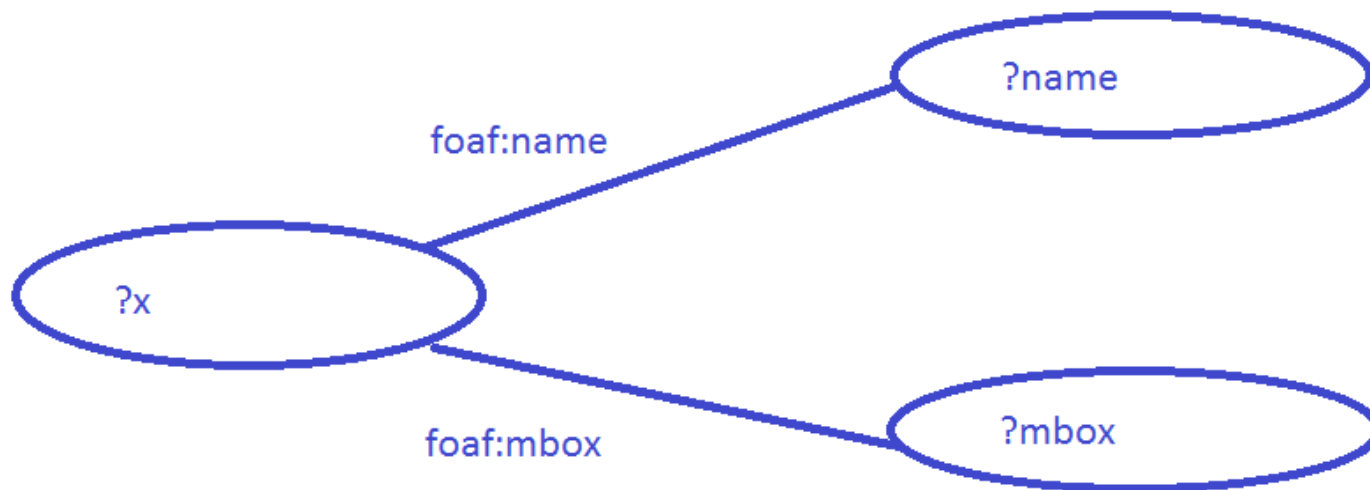
Набор данных:

```
@prefix foaf: <http://xmlns.com/foaf/0.1/> .  
_:a foaf:name "Johnny Lee Outlaw" .  
_:a foaf:mbox <mailto:jlow@example.com> .  
_:b foaf:name "Peter Goodguy" .  
_:b foaf:mbox <mailto:peter@example.org> .  
_:c foaf:mbox <mailto:carol@example.org> .
```

Запрос:

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>  
SELECT ?name ?mbox  
WHERE  
{ ?x foaf:name ?name .  
  ?x foaf:mbox ?mbox }
```

Базовый графовый шаблон: Множественные соответствия



Множественные соответствия

Каждое решение дает один способ, которым выбранные переменные могут быть привязаны к RDF терминам так, что шаблон запроса совпадает с данными.

Результирующее множество дает все возможные решения.

В приведенном выше примере, следующие два подмножества данных дают два соответствия..

`_:a foaf:name "Johnny Lee Outlaw" .`

`_:a foaf:box <mailto:jlow@example.com> .`

`_:b foaf:name "Peter Goodguy" .`

`_:b foaf:box <mailto:peter@example.org> .`

Это [basic graph pattern match](#);

все переменные, используемые в шаблоне запроса должны быть связаны в каждом решении.

Последовательность решений

Name	mbox
"Johnny Lee Outlaw"	<mailto:jlow@example.com>
"Peter Goodguy"	<mailto:peter@example.org>

А что будет результатом такого запроса?

Набор данных:

```
@prefix foaf: <http://xmlns.com/foaf/0.1/> .  
_:a foaf:name "Johnny Lee Outlaw" .  
_:a foaf:mbox <mailto:jlow@example.com> .  
_:b foaf:name "Peter Goodguy" .  
_:b foaf:mbox <mailto:peter@example.org> .  
_:c foaf:mbox <mailto:carol@example.org> .
```

Запрос:

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>  
SELECT ?x ?name ?mbox  
WHERE  
{ ?x foaf:name ?name .  
  ?x foaf:mbox ?mbox }
```

Множественные шаблоны троек: проход по графу

Найдите мне домашнюю страницу кого-нибудь, кого знает Тим Бернерс-Ли.

PREFIX foaf: <<http://xmlns.com/foaf/0.1/>>

PREFIX card: <<http://www.w3.org/People/Berners-Lee/card#>>

SELECT ?homepage

FROM <<http://dig.csail.mit.edu/2008/webdav/timbl/foaf.rdf>>

WHERE { card:i foaf:knows ?known .

?known foaf:homepage ?homepage . }

Ключевое слово **FROM** позволяет указать целевой граф в самом запросе.

Используя **?known** в качестве объекта в одной тройке и субъекта в другой, мы проходим по нескольким ребрам в графе.



Фильтры SPARQL

Ограничения ***FILTER*** используют **логические условия** для удаления нежелательных результатов запроса.

Установление соответствия с шаблоном графа дает последовательность решений, где каждое решение имеет набор связываний переменных с RDF термами.

SPARQL фильтры выбирают те решения, для которых выражение фильтра оценивается как **TRUE**.

Пример FILTER

Тестовый набор данных:

@prefix dc: <http://purl.org/dc/elements/1.1/> .

@prefix : <http://example.org/book/> .

@prefix ns: <http://example.org/ns#> .

:book1 dc:title "SPARQL Tutorial" .

:book1 ns:price 42 .

:book2 dc:title "The Semantic Web" .

:book2 ns:price 23 .

FILTER(арифметическое выражение)

SPARQL-фильтры могут ограничить при помощи арифметических выражений.

Запрос:

PREFIX dc: <http://purl.org/dc/elements/1.1/>

PREFIX ns: <http://example.org/ns#>

SELECT ?title ?price

WHERE { ?x ns:price ?price .

FILTER (?price < 30.5)

?x dc:title ?title . }

title	price
"The Semantic Web"	23

FILTER применимо к арифметическим выражениям

FILTER(для строк)

Функции SPARQL FILTER, такие как [regex](#) (регулярное выражение) могут проверить RDF литералы.

[regex](#) соответствует только строковым литералам.

[regex](#) может использоваться, чтобы соответствовать лексическим формам других литералов, используя функцию [str](#)

Запрос:

```
PREFIX dc: <http://purl.org/dc/elements/1.1/>
```

```
SELECT ?title
```

```
WHERE { ?x dc:title ?title
```

```
    FILTER regex(?title, "^SPARQL")
```

```
}
```

Title
"SPARQL Tutorial"

Встроенные в SPARQL функции фильтрации

Logical: !, &&, ||

Math: +, -, *, /

Comparison: =, !=, >, <, IN, NOT IN...

SPARQL tests: isURI, isBlank, isLiteral, isNumeric, bound

SPARQL accessors: str, lang, datatype

Other: sameTerm, langMatches, regex, REPLACE

Conditionals (SPARQL 1.1): IF, COALESCE, EXISTS, NOT EXISTS

Constructors (SPARQL 1.1): URI, BNODE, STRDT, STRLANG, UUID, STRUUID

Strings (SPARQL 1.1): STRLEN, SUBSTR, UCASE, LCASE, STRSTARTS, STRENDS, CONTAINS, STRBEFORE, STRAFTER, CONCAT, ENCODE_FOR_URI

More math (SPARQL 1.1): abs, round, ceil, floor, RAND

Date/time (SPARQL 1.1):

now, year, month, day, hours, minutes, seconds, timezone, tz

Hashing (SPARQL 1.1): MD5, SHA1, SHA256, SHA384, SHA512

Границы FILTER

Ограничение, выражаемое ключевым словом FILTER, является ограничением на решения по всей группе шаблонов, в которой появляется фильтр.

Все следующие шаблоны имеют одинаковые решения:

```
WHERE { ?x foaf:name ?name .  
        ?x foaf:mbox ?mbox .  
        FILTER regex(?name, "Smith")  
}
```

```
WHERE { FILTER regex(?name, "Smith")  
        ?x foaf:name ?name .  
        ?x foaf:mbox ?mbox .  
}
```

```
WHERE { ?x foaf:name ?name .  
        FILTER regex(?name, "Smith")  
        ?x foaf:mbox ?mbox .  
}
```

Шаблоны Optional

Дополнительные части шаблона графа могут быть заданы с помощью ключевого слова **OPTIONAL**, применяемого к шаблону графа:

При сравнении шаблонов в optional, либо **optional graph pattern** соответствует графу набора данных,

Тем самым определяя и **добавляя связываний** к **одному или нескольким** решениям,

или **оставляет решение неизменным**, не добавляя никаких новых связываний.

Шаблоны Optional

Тестовый набор данных:

@prefix foaf: <http://xmlns.com/foaf/0.1/> .

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

_:a rdf:type foaf:Person .

_:a foaf:name "Alice" .

_:a foaf:mbox <mailto:alice@example.com> .

_:a foaf:mbox <mailto:alice@work.example> .

_:b rdf:type foaf:Person .

_:b foaf:name "Bob" .

Шаблоны Optional

Запрос:

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>  
SELECT ?name ?mbox  
WHERE { ?x foaf:name ?name .  
        OPTIONAL { ?x foaf:mbox ?mbox }  
}
```

Optional patterns

name	mbox
"Alice"	<mailto:alice@example.com>
"Alice"	<mailto:alice@work.example >
"Bob"	

Множественные шаблоны графа Optional

В примере выше, только один шаблон тройки приведен в **OPTIONAL** части запроса, но, в общем случае, **OPTIONAL** может быть любой шаблон графа.

Шаблон графа может иметь ноль или более **OPTIONAL** графовых шаблонов, и любая часть шаблона запроса может иметь часть **OPTIONAL**.

В следующем примере, есть два графовых шаблона **OPTIONAL**.

Множественные шаблоны графа Optional

Тестовый набор данных:

@prefix foaf: <http://xmlns.com/foaf/0.1/> .

_:a foaf:name "Alice" .

_:a foaf:homepage <http://work.example.org/alice/> .

_:b foaf:name "Bob" .

_:b foaf:mbox <mailto:bob@work.example> .

Запрос:

PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?name ?mbox ?hpage

WHERE { ?x foaf:name ?name .

OPTIONAL { ?x foaf:mbox ?mbox } .

OPTIONAL { ?x foaf:homepage ?hpage }

}

Множественные шаблоны графа Optional

name	mbox	hpage
"Alice"		<http://work.example.org/alice/>
"Bob"	<mailto:bob@work.example>	

Альтернативы сравнения (UNION)

SPARQL обеспечивает возможность объединения шаблонов графа так, что только один из нескольких альтернативных графовых шаблонов может совпадать с набором данных.

Если более чем один из вариантов соответствует набору данных, берутся возможные шаблоны решения.

Альтернативы шаблонов синтаксически указываются с помощью ключевого слова **UNION**.

Альтернативы сравнения (UNION)

Тестовый набор данных:

@prefix dc10: <http://purl.org/dc/elements/1.0/> .

@prefix dc11: <http://purl.org/dc/elements/1.1/> .

_:a dc10:title "SPARQL Query Language Tutorial" .

_:a dc10:creator "Alice" .

_:b dc11:title "SPARQL Protocol Tutorial" .

_:b dc11:creator "Bob" .

_:c dc10:title "SPARQL" .

_:c dc11:title "SPARQL (updated)" .

Запрос:

PREFIX dc10: <http://purl.org/dc/elements/1.0/>

PREFIX dc11: <http://purl.org/dc/elements/1.1/>

SELECT ?title

WHERE { { ?book dc10:title ?title } UNION { ?book dc11:title ?title } }

Альтернативы сравнения (UNION)

title
"SPARQL Protocol Tutorial"
"SPARQL"
"SPARQL (updated) "
"SPARQL Query Language Tutorial"

Последовательности решений и модификаторы

- Шаблоны запросов генерируют неупорядоченную коллекцию решений, каждое решение является частичной функцией, отображающей переменные в RDF термины.
- Эти решения, затем рассматриваются как последовательность (последовательности) решений,
- первоначально в произвольном порядке;
- любые модификаторы последовательности затем применяются для создания другой последовательности.
- Наконец, эта последняя последовательность используется для генерации одного из результатов формы запроса SPARQL.

Модификатором последовательности решений
является один из следующих:

Модификатор Order: упорядочивает решения

Модификатор Distinct : гарантирует, что все
решения последовательности уникальны

Модификатор Offset : контролирует, где
начинаются выборка решений в общей
последовательности решений

Модификатор Limit : ограничивает число
решений

Дубликаты решений

Тестовый набор данных:

```
@prefix foaf: <http://xmlns.com/foaf/0.1/> .  
_:x foaf:name "Alice" .  
_:x foaf:mbox <mailto:alice@example.com> .  
_:y foaf:name "Alice" .  
_:y foaf:mbox <mailto:asmith@example.com> .  
_:z foaf:name "Alice" .  
_:z foaf:mbox <mailto:alice.smith@example.com> .
```

Запрос:

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>  
SELECT ?name  
WHERE { ?x foaf:name ?name }
```

Дубликаты решений

name
"Alice"
"Alice"
"Alice"

Элиминация дубликатов решений (DISTINCT)

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>  
SELECT DISTINCT ?name  
WHERE {  
  ?x foaf:name ?name  
}
```

name
"Alice"

LIMIT

Высказывание **LIMIT** задает верхнюю границу на число возвращаемых решений.

```
PREFIX foaf:  
<http://xmlns.com/foaf/0.1/>  
SELECT ?name  
WHERE { ?x foaf:name ?name }  
LIMIT 20
```

OFFSET

- **OFFSET** заставляет созданные решения начинать со сдвигом после указанного числа решений.
- Нулевое **OFFSET** ни на что не влияет.
- Использование **LIMIT** и **OFFSET** , для выбора различных подмножеств решений запроса **не будет полезным**, если порядок не будет предсказуемым с помощью **ORDER BY**.

PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?name

WHERE { ?x foaf:name ?name }

ORDER BY ?name

LIMIT 5

OFFSET 10

Формы запросов

SPARQL имеет четыре формы запросов.

Эти формы запросов используют решения, полученные от сопоставления с шаблоном для формирования результирующих наборов или RDF графов.

Формами запросов являются:

SELECT возвращает результат в виде таблицы все или подмножество переменных, связанных при сопоставлении данных с шаблоном запроса.

CONSTRUCT возвращает **граф RDF, построенный путем подстановки значений переменных в набор шаблонов троек.**

ASK возвращает **булево значение**, показывающее, соответствует ли шаблон запроса набору данных или нет.

DESCRIBE возвращает граф RDF, описывающий найденные ресурсы.

CONSTRUCT

Форма запроса CONSTRUCT возвращает один RDF граф, заданный шаблоном графа.

Результатом является RDF граф, образованный путем

- а) взятия каждого решения запроса в последовательности решений,
- б) подстановки найденных переменных в шаблон графа,
- с) объединением троек в одном графе RDF операцией объединения множеств

CONSTRUCT

Тестовый набор данных:

@prefix foaf: <http://xmlns.com/foaf/0.1/> .

_:a foaf:name "Alice" .

_:a foaf:mbox <mailto:alice@example.org> .

Запрос

PREFIX foaf: <http://xmlns.com/foaf/0.1/>

PREFIX vcard: <http://www.w3.org/2001/vcard-rdf/3.0#>

CONSTRUCT { <http://example.org/person#Alice> vcard:FN
?name }

WHERE { ?x foaf:name ?name }

CONSTRUCT

создает свойства VCard из информации FOAF:

```
@prefix vcard: <http://www.w3.org/2001/vcard-rdf/3.0#> .  
<http://example.org/person#Alice> vcard:FN "Alice" .
```

ASK (имеется ли решение?)

Тестовый набор данных:

```
@prefix foaf: <http://xmlns.com/foaf/0.1/> .  
_:a foaf:name "Alice" .  
_:a foaf:homepage <http://work.example.org/alice/> .  
_:b foaf:name "Bob" .  
_:b foaf:mbox <mailto:bob@work.example> .
```

Запрос:

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>  
ASK { ?x foaf:name "Alice" }
```

true

ASK

PREFIX foaf: <http://xmlns.com/foaf/0.1/>

ASK { ?x foaf:name "Alice" ;

foaf:mbox <mailto:alice@work.example> }

false

DESCRIBE

Форма DESCRIBE возвращает один результирующий RDF граф, содержащий RDF данные о ресурсах.

Эти данные не предписаны запросом SPARQL , а определяются **процессором запросов SPARQL**.

1) DESCRIBE <http://example.org/>

2) PREFIX foaf: <http://xmlns.com/foaf/0.1/>

DESCRIBE ?x

WHERE { ?x foaf:mbox <mailto:alice@org> }